Discussion of Exercise Sheet 6

Exercise 6.3 *Introduction to dynamic programming* **(2 bonus points)**.

Consider the recurrence

$$A_1 = 1$$
$$A_2 = 2$$
$$A_3 = 3$$
$$A_4 = 4$$
$$A_n = A_{n-1} + A_{n-3} + 2A_{n-4} \text{ for } n \geq 5.$$

(a) Provide a recursive function (using pseudo code) that computes A_n for $n \in \mathbb{N}$. You do not have to argue correctness.

Algorithm 1 $A(n)$

```
if n ≤ 4 then
    return n
else
    return A(n − 1) + A(n − 3) + 2A(n − 4)
```

(b) Lower bound the run time of your recursion from (a) by $\Omega(C^n)$ for some constant $C > 1$.

Assume n is divisible by 4

Number of calls: $T(n) = T(n-1) + T(n-3) + 2T(n-4)$ for $n \geq 5$

$\geq 2T(n-4)$

$\geq 2 \cdot 2\, T(n-8)$

$\geq 2\ 2\ 2 \cdot T(n-12)$

$\geq 2^{\frac{n}{4}}\ T(1)$

$\geq 2^{\frac{n}{4}} = (2^{\frac{1}{4}})^n \geq \Omega(C^n)$ for $C = 2^{\frac{1}{4}} \approx 1.2$

(c) Improve the run time of your algorithm using memoization. Provide pseudo code of the improved algorithm and analyze its run time. You do not need to prove correctness of the algorithm.

Algorithm 2 Compute A_n using memoization

```
memory ← 1-dimensional array of length n, filled with -1
function A_MEM(n)
    if memory[n] ≠ −1 then                          ▷ If A_n is already computed.
        return memory[n]                            } memoization
    if n ≤ 4 then
        (memory[n] ← n) unnecessary                 } base case
        return n
    else
        A_n ← A_Mem(n − 1) + A_Mem(n − 3) + 2A_Mem(n − 4)   } recursion
        memory[n] ← A_n
        return A_n
```

When calling A_Mem(n), each A_k for $1 \leq k \leq n$ is computed at most once and then stored in memory. Thus the run time of A_Mem(n) is $\Theta(n)$.

(d) Compute A_n using bottom-up dynamic programming and state the run time of your algorithm. In your solution, address the following aspects:

1. *Dimensions of the DP table*: What are the dimensions of the DP table?
2. *Subproblems*: What is the meaning of each entry?
3. *Recursion*: How can an entry of the table be computed from previous entries? Justify why your recurrence relation is correct. Specify the base cases of the recursion, i.e., the cases that do not depend on others.
4. *Calculation order*: In which order can entries be computed so that values needed for each entry have been determined in previous steps?
5. *Extracting the solution*: How can the solution be extracted once the table has been filled?
6. *Running time*: What is the running time of your solution?

Dimension: DP[1 .. n]

Teilproblem: DP[i] := A_i

Base Case: DP[1] = 1, DP[2] = 2, DP[3] = 3, DP[4] = 4

Rekursion: DP[i] = DP[i-1] + DP[i-3] + 2 DP[i-4] für $5 \leq i \leq n$

Korrektheit folgt direkt aus der Definition von A_k

Berechnungsreihenfolge: for i = 5.. n: berechne DP[i]

Lösung: DP[n]

Laufzeit: $\Theta(n)$

Exercise 6.3 *Introduction to dynamic programming* **(1 point)**.

Consider the recurrence

$$A_1 = 1$$
$$A_2 = 2$$
$$A_3 = 3$$
$$A_4 = 4$$
$$A_n = A_{n-1} + A_{n-3} + 2A_{n-4} \text{ for } n \geq 5.$$

(a) Provide a recursive function (using pseudo code) that computes A_n for $n \in \mathbb{N}$. You do not have to argue correctness.

Solution:

Algorithm 1 $A(n)$

```
if n ≤ 4 then
    return n
else
    return A(n − 1) + A(n − 3) + 2A(n − 4)
```

(b) Lower bound the run time of your recursion from (a) by $\Omega(C^n)$ for some constant $C > 1$.

(c) Improve the run time of your algorithm using memoization. Provide pseudo code of the improved algorithm and analyze its run time.

Solution:

Algorithm 2 Compute A_n using memoization

```
memory ← n-dimensional array filled with (−1)s
function A_Mem(n)
    if memory[n] ≠ −1 then                ▷ If A_n is already computed.
        return memory[n]
    if n ≤ 4 then
        memory[n] ← n
        return n
    else
        A_n ← A_Mem(n − 1) + A_Mem(n − 3) + 2A_Mem(n − 4)
        memory[n] ← A_n
        return A_n
```

(d) Compute A_n using bottom-up dynamic programming and state the run time of your algorithm. Address the following aspects in your solution:

- **Dimensions of the DP table:** The DP table is linear, its size is n.
- **Definition of the DP table:** $DP[k]$ contains A_k for $1 \leq k \leq n$.
- **Calculation of an entry:** Initialize $DP[1]$ to 1, $DP[2]$ to 2, $DP[3]$ to 3 and $DP[4]$ to 4. The entries with $k \geq 5$ are computed by $DP[k] = DP[k-1] + DP[k-3] + 2DP[k-4]$.
- **Calculation order:** We can calculate the entries of DP from smallest to largest.
- **Reading the solution:** All we have to do is read the value at $DP[n]$.
- **Run time:** Each entry can be computed in time $\Theta(1)$, so the run time is $\Theta(n)$.

→ 2-3-Tree ex

Subset Sum (≠ Subarray Sum)

Gegeben: Array $A[1, .., n]$ und target b

Gesucht: Eine Teilmenge von Indices $I \subseteq \{1, .., n\}$ sodass $\sum_{i \in I} A[i] = b$

Dimension: $DP[0 \dots n][0 \dots b]$

Teilproblem:
$$DP[i, s] := \begin{cases} 1 & \text{falls es eine Teilmenge von Indices } I \subseteq \{1, .., i\} \text{ gibt, sodass } \sum_{j \in I} A[j] = s \\ 0 & \text{sonst} \end{cases}$$

Base Case:
$DP[0,0] = 1$ → keine Elemente summieren zu 0 ✓

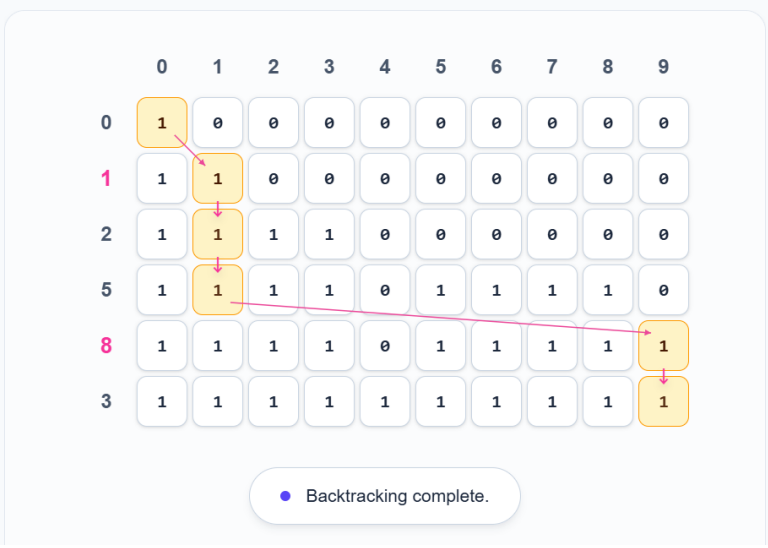
$DP[0,s] = 0$ für $1 \leq s \leq n$ → keine Elemente können nicht zu $s \geq 1$ summieren

Rekursion:
$$DP[i,s] = \underbrace{DP[i-1, s]}_{\text{wir brauchen } A[i] \text{ nicht und können ohne } A[i] \text{ zu } s \text{ summieren}} \vee \underbrace{DP[i-1,\ s-A[i]]}_{\text{wir brauchen } A[i]. \text{ Dann muss es eine Teilmenge der ersten } i-1 \text{ Elemente geben, die zu } s-A[i] \text{ summieren}} \quad \text{für } 1 \leq i \leq n, 0 \leq s \leq b$$

$:= 0$ falls $s - A[i] < 0$

Berechnungsreihenfolge:
```
for i = 1 .. n
    for s = 0 ... b:
        berechne DP[i, s]
```

Lösung: $DP[n, b]$ sagt uns, ob eine Lösung existiert. Falls ja, können wir die gesuchte Teilmenge der Elemente durch backtracking finden



Bottom-Up | Top-Down

SUBSET SUM $\mathcal{O}(n \cdot target)$

```
SUBSET-SUM(A, target)
  initialize DP[0...n][0...target]
  DP[0...n][0] = 1
  DP[0][1...target] = 0
  for i ← 1,...,n do
    for s ← 1,...,target do
      DP[i][s] ← DP[i-1][s]
      if A[i] ≤ s
        DP[i][s] ← DP[i][s] or DP[i-1][s - A[i]]
  return DP[n][target]
```

Laufzeit: $\Theta(n \cdot b)$ Bsp. $b = 4$, $b = n^2$, $b = n^{37} \log n$

Pseudopolynomiell: Ob der Algorithmus in polynomieller oder exponentieller Zeit läuft, hängt von b ab Bsp $b = 2^n$, $b = 3^n$

Anmerkung zur Notation: $DP[0 \dots n][0 \dots b] = DP[0 \dots n, 0. b] = (n+1) \times (b+1)$ Array

$DP[i,s] = DP[i][s] = DP(i,s) = T[i,s] = memo[i,s]$

$$DP[i,s] := \begin{cases} 1 & \text{falls } A \\ 0 & \text{sonst} \end{cases} = \begin{cases} \text{true} & \text{falls } A \\ \text{false} & \text{sonst} \end{cases} = \begin{cases} 1 & \text{falls } A \\ 0 & \text{falls } \neg A \end{cases}$$

Begründungssatz der Laufzeit: (für Aufgaben und Prüfung) Es gibt $(n+1)(b+1)$ DP-Einträge, wobei jeder einzelne Eintrag in $\Theta(1)$ berechnet werden kann, was zu einer Gesamtlaufzeit von $\Theta(n\,b)$ führt (Die Lösung kann in $O(1)$ ausgelesen werden)

Rucksack Problem

Gegeben: Gewichtslimit W, n items mit Gewicht $w_i \in \mathbb{N}$ und Profit $p_i \in \mathbb{N}$

Gesucht: Eine Teilmenge von Indices $I \subseteq \{1, .., n\}$ sodass das Gewichtslimit $\sum_{i \in I} w_i \leq W$ eingehalten wird und der Gesamtprofit $\sum_{i \in I} p_i$ dabei maximal ist.

Dimension: $P[0 .. n][0 .. W]$

Teilproblem: $P[i, w]$:= maximaler Profit, wenn wir nur die ersten i items betrachten und dabei das Gewichtslimit w einhalten

Base Case: $P[0, w] = 0$ für $0 \leq w \leq W$ → 0 items ergeben bei egal welchem Gewichtslimit genau 0 Profit

Rekursion:

$$P[i, w] = \max \{ \underbrace{P[i-1, w]}_{\text{:= 0 falls } w - w_i < 0}, \underbrace{P[i-1, w - w_i] + p_i}_{} \} \quad \text{für } 1 \leq i \leq n, \; 0 \leq w \leq W$$

Entweder wir nehmen item i nicht: Profit und Gewicht bleibt gleich wie bei $i-1$ items

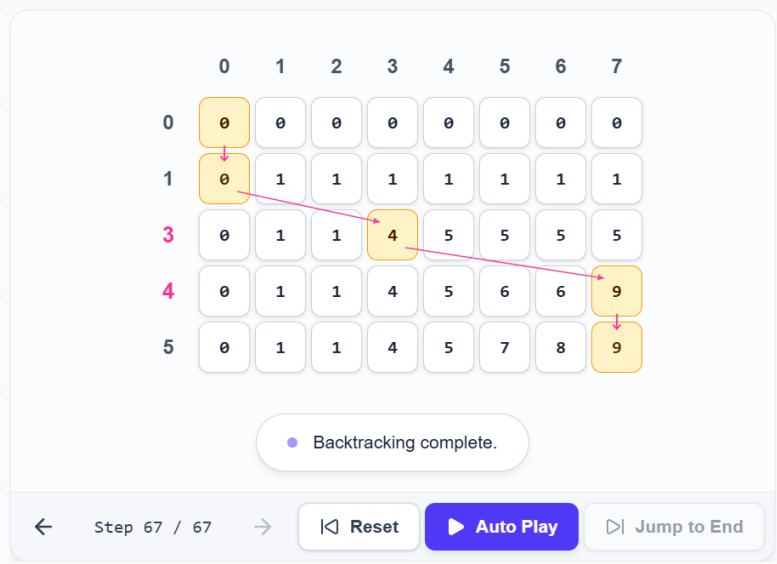
Oder wir nehmen item i: Profit erhöht sich um p_i. Die ersten $i-1$ items müssen dafür aber das Gewichtslimit $w - w_i$ einhalten

Berechnungsreihenfolge:

```
for i = 1 .. n
    for w = 0 .. W:
        berechne P[i, w]
```

Lösung: $P[n, W]$ ist der maximale Profit. Wir können die gesuchte Teilmenge der Elemente durch backtracking finden

Laufzeit: $\Theta(n W)$, pseudopolynomiell



KNAPSACK $\mathcal{O}(n \cdot W)$

```
1  KNAPSACK(W, wt, val, n)
2    initialize DP[0...n][0...W]
3    for i ← 0,...,n do
4      for w ← 0,...,W do
5        if i == 0 or w == 0
6          DP[i][w] ← 0
7        else if wt[i] ≤ w
8          DP[i][w] ← max(val[i] + DP[i-1][w-wt[i]],
8                         DP[i-1][w])
10       else
11         DP[i][w] ← DP[i-1][w]
12   return DP[n][W]
```

Alternative Lösung:

Dimension: $G[0..n][0..P]$ wobei $P = \sum_{i=1}^{n} p_i$

Teilproblem: $G[i, w]$:= minimales Gewicht, wenn wir nur die ersten i items betrachten um mindestens Profit p zu erreichen ($:= \infty$ falls nicht möglich)

Base Case: $G[0,0] = 0$

$G[0, p] = \infty$ für $1 \leq p \leq P$ → es ist unmöglich mit 0 items, Profit $p \geq 1$ zu erreichen

Rekursion:

$$G[i, p] = \min \{ \underbrace{G[i-1, p]}_{\text{wir nehmen item } i \text{ nicht}}, \overbrace{G[i-1, p-p_i] + w_i}^{:= 0 \text{ falls } p - p_i < 0} \}$$

$G[i-1, p-p_i] + w_i$: wir nehmen item i

für $1 \leq i \leq n$, $0 \leq p \leq P$

Berechnungsreihenfolge:

```
for i = 1..n
    for p = 0..P:
        berechne G[i, p]
```

Lösung: Grösstes p sodass $G[n, p] \leq W$ Wir können die gesuchte Teilmenge der Elemente durch backtracking finden

Laufzeit: $\Theta(n\,P)$, pseudopolynomiell → Approx Alg

Längste aufsteigende Teilfolge (LAT / LAS)

Gesucht: Länge einer längsten aufsteigenden Teilfolge von $A[1..n]$

Dimension: $M[1..n][1..n]$

Teilproblem: $M[i, \ell]$:= kleinstmögliche Endung einer aufsteigenden Teilfolge von $A[1..i]$ der Länge ℓ. (:= ∞ falls es keine solche Teilfolge gibt)

Base Case: $M[1,1] = A[1]$ → aufsteigende Teilfolge der Länge 1 aus $A[1]$ ist $A[1]$

$M[1,\ell] = \infty$ für $2 \leq \ell \leq n$ → es ist unmöglich mit nur einem Element eine Teilfolge der Länge $\ell \geq 2$ zu haben

Rekursion:

$$M[i,\ell] = \begin{cases} \min\{M[i-1,\ell], A[i]\} & \text{falls } \ell = 1 \text{ oder } M[i-1,\ell-1] < A[i] \\ M[i-1,\ell] & \text{sonst} \end{cases}$$

($M[i-1,\ell]$: wir benutzen $A[i]$ nicht; $A[i]$: wir hängen $A[i]$ an)

→ wir können $A[i]$ anhängen

für $2 \leq i \leq n, 1 \leq \ell \leq n$

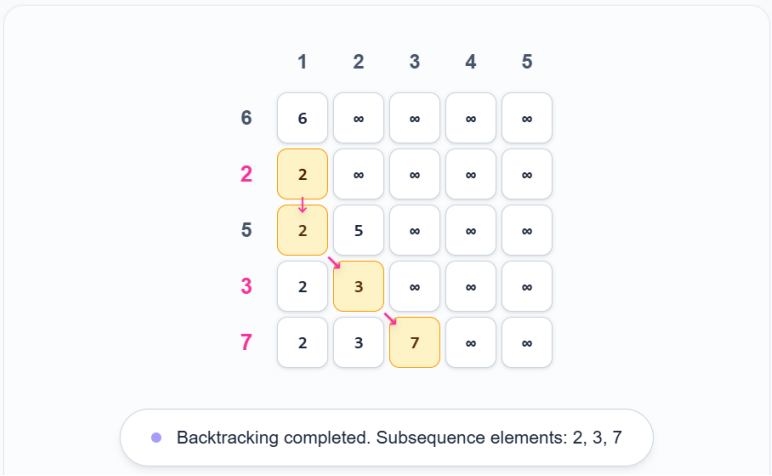
Berechnungsreihenfolge:

```
for i = 2 .. n
    for ℓ = 1 .. n:
        berechne M[i, ℓ]
```

Lösung: Grösstes ℓ sodass $M[n,\ell] < \infty$

Laufzeit: $\Theta(n^2)$

→ Lecture



LAS $\mathcal{O}(n^2)$

```
1  LAS(A)
2    DP[1][1] ← A[1]; DP[1][2...n] = ∞
3    for i ← 2,...,n do
4      for l ← 1,...,n do
5        exclude ← DP[i-1][l]
6        include ← ∞
7        if l = 1 or DP[i-1][l-1] < A[i]
8          include ← A[i]
9        DP[i][l] ← min(exclude, include)
10   return max l where DP[n][l] < ∞
```

Verbesserung in $O(n \log n)$ mit Binary Search:

Idee: es verändert sich nur ein DP-Eintrag pro Reihe

Bsp. $A = [3, 5, 4, 8, 9, 7, 2, 6]$

$3, \infty, \infty, \infty, \infty, \infty, \infty, \infty$

$B = [4, 6, 2, 7, 5, 1, 3, 9]$

$4 \quad \infty \quad \infty \quad \infty \quad \infty \quad \infty \quad \infty \quad \infty$

Wie Teilproblem finden?

1) Kategorie bestimmen:

- Optimierungsproblem (Bsp MSS, Jump Game, Edit distance, LAT, Rucksack)
  - → Was ist der maximale / minimale / kleinste / grösste / kürzeste / längste ...
- Entscheidungsproblem (Bsp Subset Sum)
  - → Gibt es eine Teilmenge / Teilarray / Teilfolge .. sodass ...
- Berechnungsproblem (Bsp Fibonacci)
  - → Was ist die Anzahl / das Ergebnis, wenn ...

2.) zusätzliche Bedingungen hinzufügen, die das Problem einfacher machen:

- Für Arrays, Strings, Mengen, und alles was man indexieren kann:
  - → ... wenn wir nur die ersten i Elemente betrachten
  - → ... wenn wir nur die ersten i Elemente betrachten, und Element i zwingend gewählt werden muss
  - → ... wenn wir bei 1 beginnen
  - → ... wenn wir bei 1 beginnen, und Element i zwingend gewählt werden muss
  - → ... wenn wir bei 1 beginnen und bei j enden
- Für Geld, Zeit, Tickets, und alles was man ausgeben kann:
  - → ... und wir mit t Geld/Zeit/Tickets starten
- Für Grenzen und Limits, die man einhalten muss:
  - → ... und wir dabei das Zeit-/Gewichts/-Limit L einhalten

Aufgaben:

You are walking up a stair. You can can take either 1 step or 2 steps at a time. How many different ways can you climb a stair of height n.

Dimension:

Teilproblem:

Base Case:

Rekursion:

Berechnungsreihenfolge:

Lösung:

Laufzeit:

You want to choose a new pin for your computer. You are allowed to use up to n digits, where each digit is between 0 and 9. How many distinct PINs can you create?

Dimension:

Teilproblem:

Base Case:

Rekursion:

Berechnungsreihenfolge:

Lösung:

Laufzeit:

You want to build a vertical tower of exact height n units. You have an unlimited supply of three types of bricks:

- **Size 1 bricks** (height of 1 unit)
- **Size 2 bricks** (height of 2 units)
- **Size 4 bricks** (height of 4 units)

Two towers are considered **distinct** if the sequence of brick sizes stacked from bottom to top differs. For example, placing a 1-brick then a 2-brick is different from placing a 2-brick then a 1-brick.

Find the total number of distinct ways to build a tower of height n.

Dimension:

Teilproblem:

Base Case:

Rekursion:

Berechnungsreihenfolge:

Lösung:

Laufzeit:

~ 20 min Zeit

/ 8 P von 100 Punkten

Theory Task T3.

An array of non-negative integers $A = [a_1, \ldots, a_n]$ is called *summy* if and only if, for all $i \in \{2, \ldots, n\}$, there exists a (possibly empty) set $I \subseteq \{1, \ldots, i-1\}$ such that $a_i = \sum_{j \in I} a_j$. In other terms, every integer in the array except the first one must be the sum of (distinct) integers that precede it in the array.

For example,

- The array $[2, 2, 4, 6, 0, 12]$ is summy, because $2 = 2$, $4 = 2 + 2$, $6 = 2 + 4$, $12 = 2 + 4 + 6$.
- The array $[2, 2, 4, 6, 0, 13]$ is not summy, since 13 can not be written as a sum of integers from $\{2, 2, 4, 6, 0\}$.

Provide a *dynamic programming* algorithm that, given an array A of length n, returns `True` if the array is summy, and `False` otherwise. In order to obtain full points, your algorithm should have an $O(n \cdot \max A)$ runtime (where $\max A$ means the maximum value of entries in A). Address the following aspects in your solution:

↳ Hinweis für die Dimension

1) *Definition of the DP table*: What are the dimensions of the table $DP[\ldots]$? What is the meaning of each entry ?
2) *Computation of an entry*: How can an entry be computed from the values of other entries ? Specify the base cases, i.e., the entries that do not depend on others.
3) *Calculation order*: In which order can entries be computed so that values needed for each entry have been determined in previous steps ?
4) *Extracting the solution*: How can the final solution be extracted once the table has been filled ?
5) *Running time*: What is the running time of your algorithm ? Provide it in Θ-notation in terms of n and $\max A$, and justify your answer.

↳ immer wenn ihr das lest, müsst ihr mindestens einen Satz schreiben

Dimension:

Teilproblem:

Base Case:

Rekursion:

Berechnungsreihenfolge:

Lösung:

Laufzeit:

→ Ex 7.1

You are a history student, "Alex," standing at the entrance of the "Grand Boulevard of Museums" (museum 0). You have a research paper due tomorrow, and your goal is to acquire the maximum amount of "knowledge" possible.

You have two limited resources:

- A starting budget of M dollars.
- A total time limit of T hours.

The boulevard has N museums, numbered 0 to $N-1$. For each museum i, you are given three pieces of information:

- `ticket_cost[i]` : The cost in dollars to buy a ticket.
- `visit_time[i]` : The time in hours it takes to see the exhibit.
- `knowledge[i]` : The knowledge points (KP) you gain from the exhibit.

The Rules of the Tour

When you are at the entrance of museum i, you have two choices:

1. **Visit the Museum:**
   - You **can only** do this if you have enough money (`>= ticket_cost[i]`) AND enough time (`>= visit_time[i]`).
   - If you visit, you pay the `ticket_cost[i]` , spend the `visit_time[i]` , and gain the `knowledge[i]` .
   - After your visit, you immediately move to the front of the next museum ($i+1$).
2. **Skip the Museum:**
   - You **can only** do this if you have at least **1 hour** of time remaining.
   - Skipping costs you no money, but it costs **1 hour** of time to walk to the next museum.
   - You gain **0** knowledge.
   - You move to the front of the next museum ($i+1$).

Your tour ends when you have considered the last museum (i.e., you arrive at index N) or when you run out of time and cannot even afford to skip (i.e., you have 0 time left).

What is the absolute maximum amount of knowledge you can gain?

Example

- $N = 3$ museums
- $M = 10$ (dollars)
- $T = 9$ (hours)

Museum (i)	ticket_cost[i]	visit_time[i]	knowledge[i]
skip → 0	5	2	7
visit → 1	4	3	3
visit → 2	6	5	8

Dimension:

Teilproblem:

Base Case:

Rekursion:

Berechnungsreihenfolge:

Lösung:

Laufzeit:

Given a binary string $S \in \{0,1\}^n$ of length n, let $g(S)$ be the **positional sum** of the string. We define the positional sum as the sum of all 1-based indices i such that the i-th character of S is a '1'.

For example, if $S = \text{"}01011\text{"}$:

- The '1's are at indices 2, 4, and 5.
- $g(S) = 2 + 4 + 5 = 11$.
- For $S = \text{"}1000\text{"}$, $g(S) = 1$.
- For $S = \text{"}0000\text{"}$, $g(S) = 0$.

Given n and $\mathbf{K}$, the goal is to count the number of binary strings S of length n with $g(S) = \mathbf{K}$

Describe a DP algorithm that, given positive integers n and $\mathbf{K}$, reports the required number. Your solution should have a time complexity of at most $O(n\mathbf{K})$.

Dimension:

Teilproblem:

Base Case:

Rekursion:

Berechnungsreihenfolge:

Lösung:

Laufzeit: